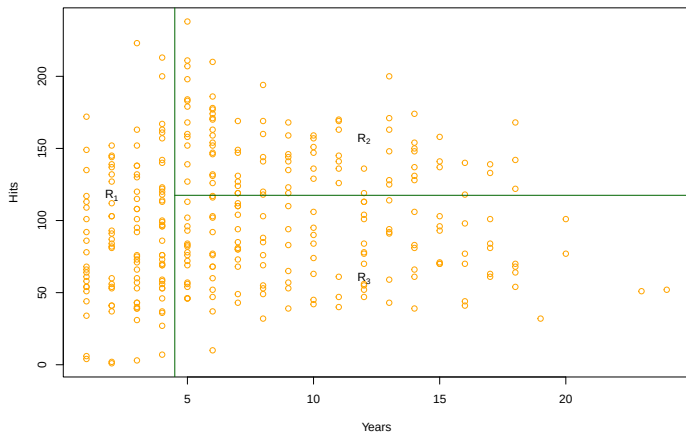


Tree-based-methods

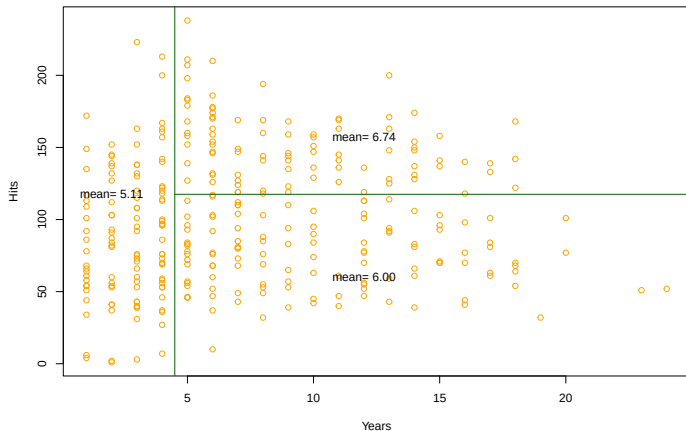
Regression and classification trees

- These methods are based on the idea of splitting the predictor space in segments and compute averages (regression) or rates (classification) of the response variable in each segment.

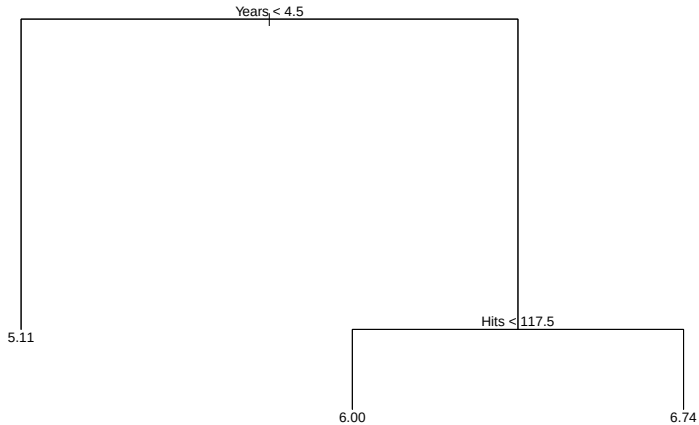


Example with Hitters data

- These methods are based on the idea of splitting the predictor space in segments and compute averages (regression) or rates (classification) of the response variable in each segment.



Regression trees



How do we stratify the predictor space?

- ▶ How are the numbers 4.5 and 117.5 determined?
- 1. Divide the predictor space, i.e. the space spanned by $X_1, \dots, X_p$, into J non-overlapping regions, $R_1, R_2, \dots, R_J$.
- 2. All observations in a region will be predicted with the same value, the average of the Y 's in the region.

Dividing the predictor space

- ▶ We will insist to make the regions (high-dimensional) rectangles. We could allow other shapes at the cost of computational complexity.
- ▶ Find rectangles, $R_1, \dots, R_J$, so that

$$RSS = \sum_{j=1}^J \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2,$$

where $\hat{y}_{R_j}$ is the average of the response in region R_j , is minimized.

Dividing the predictor space

- ▶ Not feasible to consider all possible rectangles (compare with best subset vs forward/backward stepwise selection in linear regression)
- ▶ We use recursive, binary splitting.
- ▶ A *top-down* approach since we start with all observations in one single region and split the predictor space with one variable at a time (compare with backward stepwise selection)
- ▶ The approach is *greedy* since it splits with the best variable at the present stage only.

More concretely

1. Start by considering the entire predictor space X
2. Choose the predictor X_j and the cutpoint s that splits X in two pieces

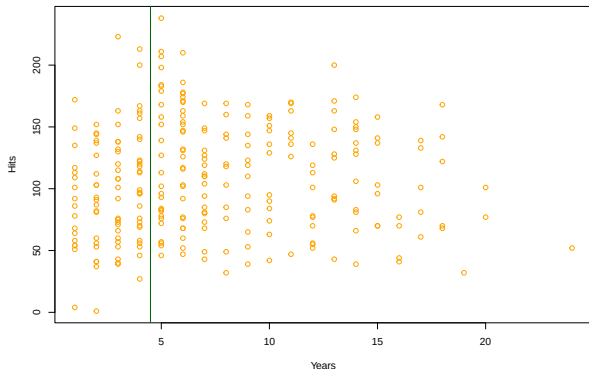
$$R_1(j, s) = \{X | X_j < s\} \text{ and } R_2(j, s) = \{X | X_j \geq s\}$$

by minimizing

$$RSS = \sum_{i: x_i \in R_1(j, s)} (y_i - \hat{y}_{R_1})^2 + \sum_{i: x_i \in R_2(j, s)} (y_i - \hat{y}_{R_2})^2$$

For the Hitters example

For illustration we only consider the two predictors $X_1 = \text{Years}$ and $X_2 = \text{Hits}$ and the response $Y = \log\text{Salary}$.



Try $X_1 = \text{Hits}$ first

$$R_1(1, s) = \{(X_1, X_2) | X_1 < s\} \text{ and } R_2(1, s) = \{(X_1, X_2) | X_1 \geq s\}$$

Find s that minimizes

$$RSS = \sum_{i: x_i \in R_1(1, s)} (y_i - \hat{y}_{R_1})^2 + \sum_{i: x_i \in R_2(1, s)} (y_i - \hat{y}_{R_2})^2$$

```
RSS=function(s,x1,y)
{
  yR1=mean(y[x1<s])
  yR2=mean(y[x1>=s])
  rss=sum((y[x1<s]-yR1)^2)+sum((y[x1>=s]-yR2)^2)
  return(rss)
}
optimise(RSS,lower=0,upper=50,x1=Hitters$Hits,y=Hitters$logSalary)
```

Try Hits first

```
RSS=function(s,x1,y)
{
  yR1=mean(y[x1<s])
  yR2=mean(y[x1>=s])
  rss=sum((y[x1<s]-yR1)^2)+sum((y[x1>=s]-yR2)^2)
  return(rss)
}
optimise(RSS,lower=0,upper=50,x1=Hitters$Hits,y=Hitters$logSalary)

## $minimum
## [1] 11.80345
##
## $objective
## [1] 203.662
```

An then $X_2 = \text{Years}$

```
optimise(RSS,lower=0,upper=50,x1=Hitters$Years,y=Hitters$logSalary)

## $minimum
## [1] 4.508575
##
## $objective
## [1] 115.0585
```

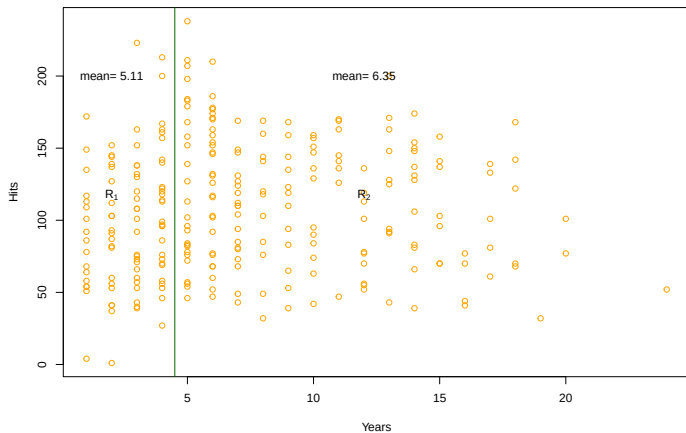
Conclusion: The minimum RSS is found when $j = 2$ and $s = 4.5$

$\implies$

The first split is made by

$$R_1 = \{(X_1, X_2) | X_2 < 4.5\} \text{ and } R_2 = \{(X_1, X_2) | X_2 \geq 4.5\}$$

After the first split



The algorithm

- ▶ Start by considering the entire predictor space X
- ▶ Choose the predictor X_j and the cutpoint s that splits X in two pieces

$$R_1(j, s) = \{X | X_j < s\} \text{ and } R_2(j, s) = \{X | X_j \geq s\}$$

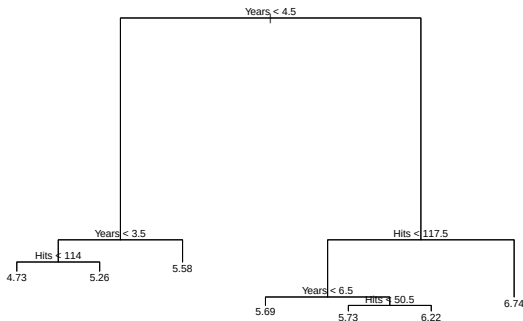
by minimizing

$$RSS = \sum_{i: x_i \in R_1(j, s)} (y_i - \hat{y}_{R_1})^2 + \sum_{i: x_i \in R_2(j, s)} (y_i - \hat{y}_{R_2})^2$$

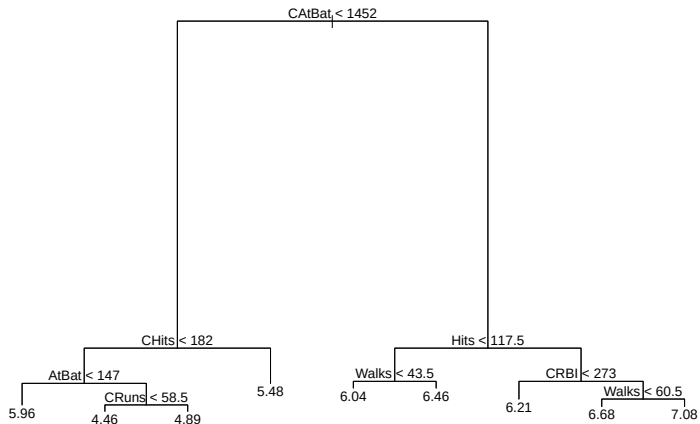
- ▶ The two regions R_1 and R_2 are now each splitted according to the same principle.
- ▶ This is repeated for each new region until a convergence criteria is met. One such criteria could be that a region is stopped being splitted when there are less than, say, 10 observations in it.

The Hitters data

```
Hitters$logSalary=log(Hitters$Salary)
require(tree)
tree=tree(logSalary~Years+Hits,data=Hitters,mincut=10)
plot(tree)
text(tree,digits = 3)
```



The Hitters data with all variables



Tree pruning

- ▶ A large tree might fit well to the training data but fail to predict the test data - overfitting!
- 1. Grow a large tree T_0
- 2. Use so called *complexity pruning* to the large tree, i.e. minimize

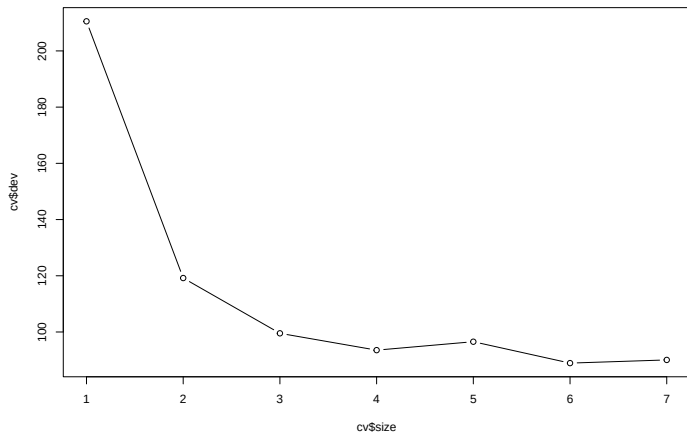
$$g(T) = \sum_{m=1}^{|T|} \sum_{i: x_i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T|,$$

where T is a tree that is a “cut” version of T_0 .

- ▶ The term $\alpha |T|$ is penalizing large trees. If two trees have identical RSS, the smallest one is considered better (compare to adjusted R^2)
- ▶ α can be determined by cross-validation

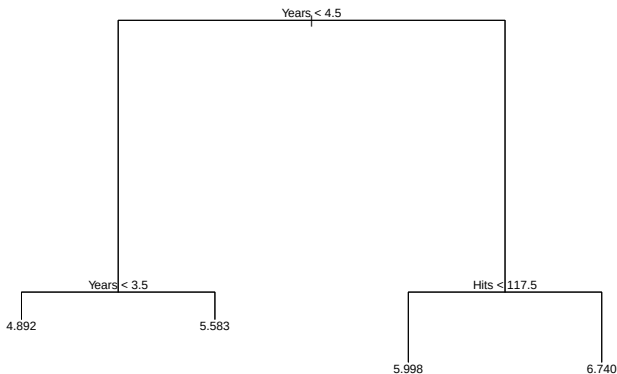
The Hitters data

```
tree=tree(logSalary~Years+Hits,data=Hitters,mincut=10)
cv=cv.tree(tree)
plot(cv$size,cv$dev,type="b")
```



The Hitters data

```
pruned.tree=prune.tree(tree,best=4)  
plot(pruned.tree)  
text(pruned.tree)
```



Classification trees

- ▶ In this Y can obtain the values $1, 2, \dots, K$
- ▶ Let us repeat the procedure for regression trees:
 1. Divide the predictor space, i.e. the space spanned by $X_1, \dots, X_p$, into J non-overlapping regions, $R_1, R_2, \dots, R_J$.
 2. All observations in each region will be predicted with the same value, the average of the Y 's in the region.
- ▶ If we amend 2. a little bit
 2. All observations in a region will be predicted by the value (out of $1, 2, \dots, K$) having the highest proportion of the observations in the region.
- ▶ we have a sensible criteria for classification.

Dividing the predictor space

- ▶ Furthermore, we need to substitute the fitting function (RSS) with something appropriate for classification.
- ▶ Let $\hat{p}_{mk}$ be the proportion of observations in R_m from class k
- ▶ Find rectangles, $R_1, \dots, R_J$, so that one of
- ▶ Classification error

$$E = 1 - \max_k(\hat{p}_{mk})$$

- ▶ Gini index

$$G = \sum_{k=1}^K \hat{p}_{mk}(1 - \hat{p}_{mk})$$

- ▶ Cross-entropy

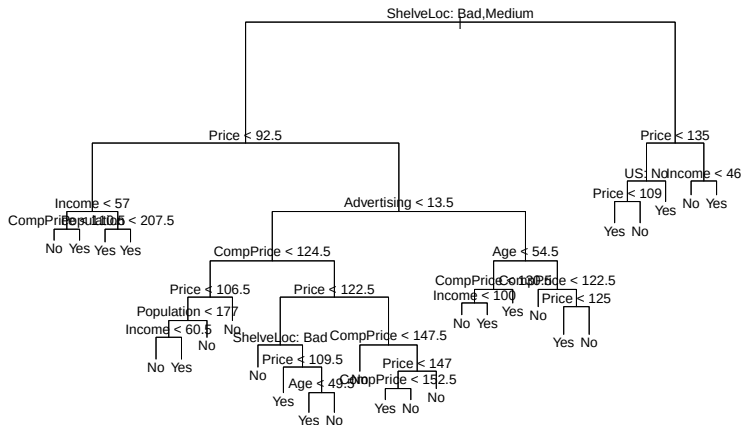
$$D = - \sum_{k=1}^K \hat{p}_{mk} \log(\hat{p}_{mk})$$

is minimized

Car seats example

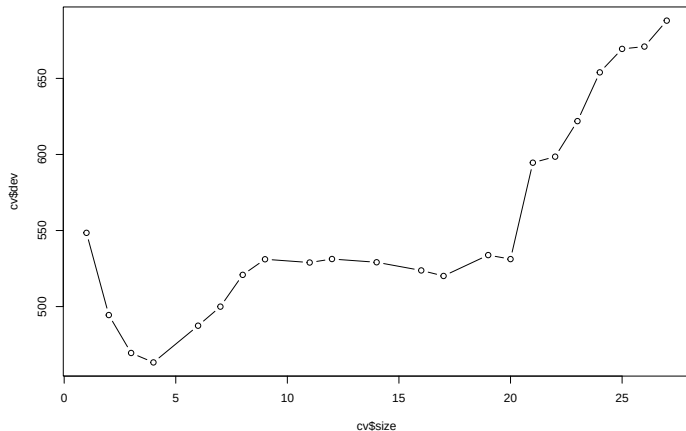
```
Carseats$High=as.factor(ifelse(Carseats$Sales<=8, "No", "Yes"))  
tree.carseats=tree(High~.-Sales,data=Carseats)  
plot(tree.carseats)  
text(tree.carseats,pretty=0)
```

Car seats example



Car seats example - pruning

```
cv=cv.tree(tree.carseats)  
plot(cv$size,cv$dev,type="b")
```



Car seats example - pruning

```
pruned.tree=prune.tree(tree.carseats,best=4)  
plot(pruned.tree)  
text(pruned.tree)
```

